

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки і комп'ютерних технологій

Звіт
про виконання лабораторної роботи №5
з курсу «Системи штучного інтелекту»
на тему:
«Пошук алгоритмом Дейкстри на зваженому графі»

Виконав:
студент групи ФЕІ-42
Трофимов Володимир
Перевірив:
ас. Іжик О. Б.

Зміст

Тема	3
Мета	3
Вхідні дані	3
Завдання	3
Критерії виконання роботи	4
Теоретичні відомості	5
1. Зважені графи та їх основні поняття.....	5
2. Алгоритм Дейкстри: суть та принцип функціонування	5
3. Суть модифікації пошуку в ширину (BFS) за Дейкстрою	6
4. Порівняльний аналіз BFS, DFS та алгоритму Дейкстри.....	7
5. Блок-схема та діаграма станів	8
5.1. Опис та аналіз блок-схеми алгоритму	8
Інтерфейс та принцип роботи програми	11
Результати дослідження роботи програми	14
<i>Дослідження 1: Базовий пошук найкоротшого шляху на схемі автомобільних шляхів України</i>	14
<i>Дослідження 2: Дзеркальна зміна напрямку пошуку (Start ↔ Target)</i>	15
<i>Дослідження 3: Видалення та додавання елементів графа (зміна розміру та порядку).</i>	16
<i>Дослідження 3.1: Видалення транзитної вершини (Вінниця)</i>	16
<i>Дослідження 3.2: Додавання прямої швидкісної магістралі (ребра)</i>	16
<i>Дослідження 4: Пошук найкоротшого шляху на орієнтованому графі (односторонній рух)</i>	17
Висновок	19
Код програми	21

Тема

Знаходження найкоротшого шляху на зважених графах за допомогою алгоритму Дейкстри. Знаходження найкоротшого шляху автомобільного сполучення між містами України.

Мета

Створити програму виконання пошуку Дейкстри на зваженому графі та дослідити особливості її роботи.

Вхідні дані

- схема автомобільних маршрутів України (рис. 1 з методичних вказівок, однак замінений на інше зображення для програми) та відстані між найбільшими містами (табл. 1 з методичних вказівок);
- зважений граф із зафіксованим положенням (координатами) вершин, що відповідають географічному розташуванню міст.
- початкова (Start) та цільова (Target) точки маршруту.
- таблиця відстаней між містами України (50 сполучень із заданими вагами ребер у кілометрах).

Завдання

Використовуючи схему автомобільних шляхів України та таблицю відстаней, побудувати зважений граф, який відображає схему автомобільних сполучень між найбільшими містами України (вершини графа — міста, ваги ребер — відстані між ними в кілометрах).

Використовуючи алгоритм Дейкстри, на одній з мов об'єктно-орієнтованого програмування високого рівня (Python) розробити програму пошуку найкоротшого шляху на зваженому графі.

У програмі реалізувати:

- візуальне графічне представлення зваженого графа на площині;
- знаходження найкоротшого шляху від заданої початкової до цільової вершини з використанням алгоритму Дейкстри;
- виведення покрокового процесу пошуку та фінального результату;
- виведення переліку міст знайденого маршруту та сумарної відстані між ними в окремому вікні інтерфейсу;
- визначення переваг, недоліків та часової/просторової складності алгоритму Дейкстри.

Критерії виконання роботи

- 1) Створити графічний інтерфейс керування роботою програми, який передбачає можливість виконання всіх необхідних для дослідження операцій, у тому числі:
 - візуалізацію топології зваженого графа (вершини з підписами міст, ребра з відображенням вагових коефіцієнтів та білою окантовкою для контрастності);
 - візуалізацію безпосереднього процесу пошуку (відображення станів вершин: невідвідані, у черзі з пріоритетом, поточна розкрита вершина, зафіксовані вершини) та підсвічування підсумкового знайденого маршруту;
 - можливість керування швидкістю анімації, покрокового виконання («Крок»), миттєвого обчислення без анімації та скидання стану пошуку;
 - можливість динамічно змінювати розмірність і порядок графа: додавати та видаляти вершини (міста), додавати, видаляти та модифікувати ребра і дуги з коригуванням їхньої ваги через елементи інтерфейсу або гарячі клавіші;
 - функцію швидкого повернення до початкової конфігурації («Reset Graph»);
 - відображення результатів пошуку (знайдений перелік міст, сумарна довжина шляху в кілометрах, кількість відкритих і відвіданих вершин, кількість ітерацій та точний час виконання) як на головній панелі, так і в окремому спливаючому вікні (звіті) та зведених таблиці експериментів.
- 2) Дослідити роботу розробленої програми та алгоритму Дейкстри:
 - знайти найкоротший шлях між заданими початковою та цільовою вершинами на базовому графі автомобільних шляхів України;
 - дослідити вплив дзеркальної зміни напрямку пошуку (взаємна заміна початкової та цільової точок);
 - дослідити вплив зміни розміру графа (видалення і додавання ребер/дуг, що входять або не входять до знайденого маршруту, зміна вагових коефіцієнтів);
 - дослідити вплив зміни порядку графа (видалення та введення вершин графа) на конфігурацію найкоротшого шляху та кількість виконаних ітерацій.
- 3) Теоретично обґрунтувати та проаналізувати особливості алгоритму:
 - пояснити, у чому полягає суть модифікації класичної схеми пошуку в ширину (BFS), яку застосував Дейкстра для пошуку найкоротшого шляху на зваженому графі;

- пояснити, як заміна звичайної черги FIFO на чергу з пріоритетом (Min-Heap) та операція релаксації ребер впливають на організацію обходу графа;
- навести блок-схему алгоритму Дейкстри.

Теоретичні відомості

1. Зважені графи та їх основні поняття

Граф — це математична структура, що складається з множини вершин (вузлів) та множини зв'язків між ними (ребер або дуг):

$$G = (V, E)$$

де:

- V — скінченна множина вершин графа ($|V| = n$ — порядок графа);
- E — множина ребер або дуг ($|E| = m$ — розмір графа).

У зваженому графі кожному ребру $(u, v) \in E$ зіставляється числове значення — вага ребра:

$$w: E \rightarrow \mathbb{R}^+$$

У прикладних задачах моделювання транспортних та логістичних мереж (зокрема мережі автомобільних сполучень України) вершини $v_i \in V$ представляють географічні пункти (міста), а ваги ребер $w(u, v)$ відповідають фізичній відстані між ними в кілометрах або тривалості проїзду.

Шляхом у зваженому графі від вершини v_0 до вершини v_k називається послідовність вершин $P = (v_0, v_1, \dots, v_k)$, де для кожного $i \in \{0, \dots, k-1\}$ існує ребро $(v_i, v_{i+1}) \in E$. Загальна вартість (довжина) зваженого шляху $L(P)$ визначається як сума ваг усіх ребер, що входять до нього:

$$L(P) = \sum_{i=0}^{k-1} w(v_i, v_{i+1})$$

Завдання знаходження найкоротшого шляху між початковою вершиною S (Start) та цільовою вершиною T (Target) полягає у визначенні такого шляху P^* , для якого:

$$L(P^*) = \min_{P \in \mathcal{P}_{S \rightarrow T}} L(P)$$

де $\mathcal{P}_{S \rightarrow T}$ — множина всіх допустимих шляхів із S у T .

2. Алгоритм Дейкстри: суть та принцип функціонування

Алгоритм Дейкстри (розроблений нідерландським ученим Едсгером Дейкстрою у 1959 році) є класичним жадібним алгоритмом пошуку найкоротших шляхів від однієї початкової вершини до всіх інших (або конкретної цільової) у зважених графах за умови невід'ємності ваг ребер:

$$\forall e \in E: w(e) \geq 0$$

Базові структури даних алгоритму

Для кожної вершини $v \in V$ підтримуються такі параметри:

- $d[v]$ (*dist / відстань*): поточна оцінка довжини найкоротшого шляху від початкової вершини S до v . На початку: $d[S] = 0$; для всіх $v \neq S$: $d[v] = \infty$.
- $parent[v]$ (*попередник*): вказівник на попередню вершину на поточному найкоротшому маршруті від S до v (служить для відновлення повного ланцюжка міст після завершення пошуку).
- *пріоритетна черга (Min-Priority Queue / Min-Heap)*: містить пари $(d[v], v)$. Вона забезпечує швидке вилучення вершини з мінімальним поточним значенням $d[v]$.
- *множина зафіксованих (розкритих) вершин U* : підмножина вершин, для яких гарантовано знайдено мінімальну відстань від старту.

Операція релаксації (Edge Relaxation)

Основою роботи алгоритму є релаксація ребер. Якщо під час розкриття вершини u виявляється, що шлях до суміжної вершини v через u є коротшим за відомий раніше, оцінка $d[v]$ зменшується:

$$\text{якщо } d[u] + w(u, v) < d[v], \quad \text{то } d[v] = d[u] + w(u, v), \quad parent[v] = u$$

Після оновлення пара $(d[v], v)$ вноситься до пріоритетної черги.

3. Суть модифікації пошуку в ширину (BFS) за Дейкстрою

Алгоритм Дейкстри можна інтерпретувати як розвиток і метричне узагальнення алгоритму пошуку в ширину (BFS).

Критерій оптимізації:

- У класичному BFS ребра вважаються незваженими (вага кожного $w = 1$), а цільова функція полягає в мінімізації кількості переходів (ребер). Хвиля пошуку поширюється дискретними рівнями: 1 крок, 2 кроки тощо.
- В алгоритмі Дейкстри враховується реальна вага ребер. Шлях з 2 ребер може бути сумарно значно довшим за шлях із 4 ребер (наприклад: пряма траса 500 км проти об'їзду $120 + 130 + 110 = 360$ км). Дейкстра оптимізує саме метричну сумарну вагу.

Організація черги:

- *BFS* використовує чергу *FIFO* (*First In, First Out*): порядок додавання суміжних вершин однозначно визначає порядок їх подальшого розкриття.
- Дейкстра замінив чергу *FIFO* на чергу з пріоритетом (*Min-Heap*). Вершини розкриваються строго за зростанням сумарної відстані від початкової точки, незалежно від того, скільки ребер довелося пройти. Хвиля пошуку поширюється концентричними колами однакової метричної вартості.

Логіка фіксації стану вершини:

- У *BFS* вершина вважається відвіданою в момент першого виявлення серед сусідів і повторно до черги не заноситься.
- У Дейкстри вершина вважається остаточно закритою лише в момент вилучення її з пріоритетної черги як глобального мінімуму. До цього моменту її оцінка $d[v]$ може багаторазово покращуватися за рахунок релаксацій з інших напрямків.

4. Порівняльний аналіз *BFS*, *DFS* та алгоритму Дейкстри

Характеристика	Пошук у ширину (<i>BFS</i>)	Пошук у глибину (<i>DFS</i>)	Алгоритм Дейкстри
Основна структура	Черга <i>FIFO</i>	Стек <i>LIFO</i>	Черга з пріоритетом (<i>Min-Heap</i>)
Тип графа	Незважений	Довільний	Зважений ($w \geq 0$)
Принцип просування	Пошарово (за рівнями)	Вглиб обраної гілки	За найменшою сумарною вагою
Гарантія найкоротшого шляху	Тільки за кількістю ребер	Не гарантує	Гарантує мінімальну сумарну вагу
Підтримка ваг ребер	Ні (всі ваги = 1)	Ні	Так (будь-які $w \geq 0$)
Часова складність	$O(V + E)$	$O(V + E)$	$O((V + E) \log V)$ (з купою)
Просторова складність	$O(V)$	$O(V)$	$O(V)$

5. Блок-схема та діаграма станів

5.1. Опис та аналіз блок-схеми алгоритму

Блок-схема відображає послідовність етапів виконання пошуку найкоротшого шляху на зваженому графі за алгоритмом Дейкстри. Процес можна структурувати за такими фазами:

1) Фаза ініціалізації:

- Зчитується топологія зваженого графа $G = (V, E)$ та ваги всіх ребер $w(u, v) \geq 0$.
- Визначаються початкова вершина *Start* та цільова вершина *Target*.
- Для кожної вершини $v \in V$ встановлюється початкове значення поточної найкоротшої відстані $d[v] = \infty$ та скидається вказівник на батьківську вершину: $parent[v] = \text{NULL}$.
- Для початкової вершини фіксується нульова відстань: $d[\text{Start}] = 0$.
- Створюється пріоритетна черга з мінімальною купою (PQ), куди заноситься початковий елемент $(0, \text{Start})$.

2) Основний ітераційний цикл пошуку:

- Робота циклу триває доти, доки черга PQ не спорожніє або доки цільова вершина не буде вилучена з черги як мінімальна.
- На кожній ітерації з купи вилучається пара $(\text{cur_dist}, u)$, яка має мінімальну накопичену відстань.
- Перевірка на застарілість (Lazy deletion): оскільки в стандартній реалізації бінарної купи вага вершини не оновлюється на місці, перевіряється умова $\text{cur_dist} > d[u]$. Якщо вона виконується, цей запис є застарілим і пропускається без обробки.
- Критерій зупинки: якщо вилучена вершина $u == \text{Target}$, алгоритм негайно завершує роботу, оскільки для неї гарантовано знайдено глобально мінімальну сумарну відстань.
- Якщо вершина не є цільовою, вона додається до множини зафіксованих (остаточно закритих) вершин.

3) Релаксація суміжних ребер:

- Для кожного суміжного сусіда v розраховується потенційна довжина шляху через u : $\text{new_dist} = d[u] + w(u, v)$.
- Якщо $\text{new_dist} < d[v]$, це свідчить про знаходження коротшого маршруту:
 - значення $d[v]$ оновлюється на new_dist ;
 - зв'язок $parent[v] = u$ перепризначається;
 - нова пара $(d[v], v)$ додається до черги PQ .

4) Фаза відновлення маршруту:

- Якщо після виходу з циклу значення $d[Target] == \infty$, робиться висновок, що шлях між заданими містами відсутній (компоненти графа незв'язні).
- Якщо $d[Target] < \infty$, виконується зворотний прохід через масив *parent*, починаючи від *Target* до *Start*. Отримана послідовність реверсується, формуючи фінальний список міст та сумарний кілометраж $d[Target]$.

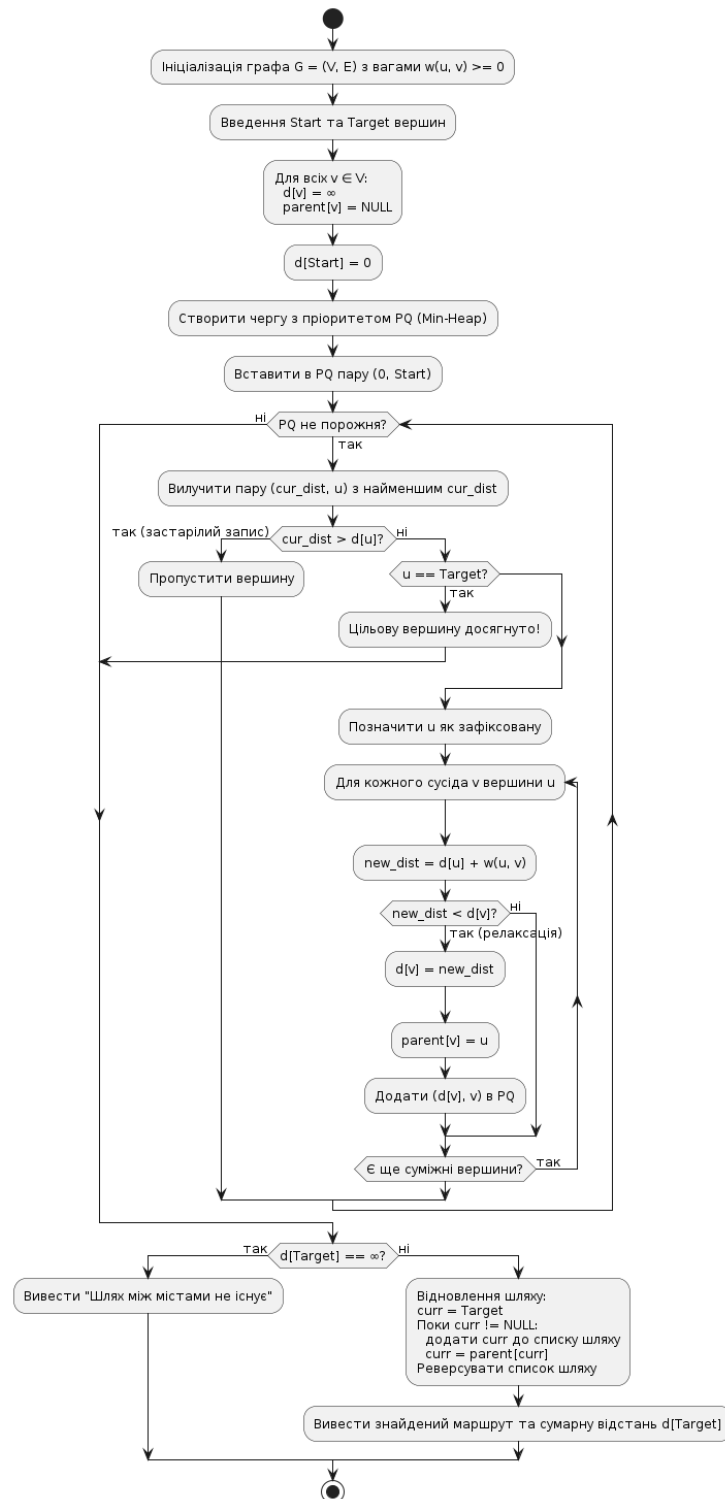


Рис.1 – Блок-схема алгоритму Дейкстри.

5.2. Опис життєвого циклу та станів вершин у процесі пошуку

У процесі виконання алгоритму Дейкстри кожна вершина графа проходить чітко визначену послідовність станів, що дає змогу візуально контролювати та інтерпретувати роботу алгоритму в графічному інтерфейсі:

- Стан «Невідвідана» (*Unvisited*): початковий стан абсолютно всіх вершин графа (крім *Start*). Відстань до них оцінюється як нескінченність ($d[v] = \infty$), а ребра до них ще не розглядалися.
- Стан «У черзі» (*InQueue* / *Candidate*): вершина потрапляє в цей стан, коли до неї знайдено хоча б один шлях від однієї з уже розкритих вершин (первинна релаксація). Вершина отримує числову оцінку $d[v] < \infty$ і заноситься в чергу з пріоритетом. У цьому стані оцінка $d[v]$ може покращуватися (зменшуватися) багаторазово, якщо з іншого напрямку буде виявлено ще коротший маршрут.
- Стан «Поточна» (*Active* / *Current*): вершина вилучається з пріоритетної черги як глобальний мінімум на поточному кроці алгоритму. Саме з неї проводиться перегляд і релаксація всіх суміжних ребер.
- Стан «Зафіксована» (*Settled* / *Expanded*): після перегляду всіх суміжних зв'язків вершина вважається остаточно закритою. Завдяки властивості невід'ємності ваг ребер гарантується, що знайдена для неї відстань $d[v]$ є строго мінімальною і змінюватися більше не буде.
- Стан «Цільова досягнута» (*TargetReached*): фінальний стан, який фіксує досягнення точки призначення та ініціює побудову підсумкового маршруту за збереженими вказівниками попередників.

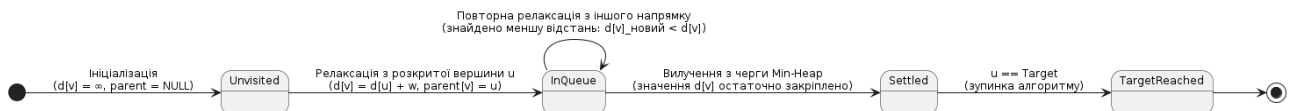


Рис.2 – Діаграма переходів станів вершини в процесі пошуку.

Інтерфейс та принцип роботи програми

Графічний інтерфейс розробленої програми реалізовано мовою Python з використанням бібліотеки CustomTkinter, яка забезпечує створення сучасних елементів графічного інтерфейсу та їх взаємодію з алгоритмічною частиною програми. Структуру інтерфейсу організовано у вигляді трьох основних функціональних областей: панелі навігації між вкладками, панелі керування параметрами графа-лабіринту та алгоритму однонапрявленого пошуку і області візуалізації графа та процесу пошуку.

Основна функціональність програми зосереджена у вкладці «Лабораторна», яка призначена для виконання експериментів та дослідження особливостей алгоритму Дейкстри. Додатково передбачено вкладки «Результати» та «Теорія Дейкстри» (див. рис.3).

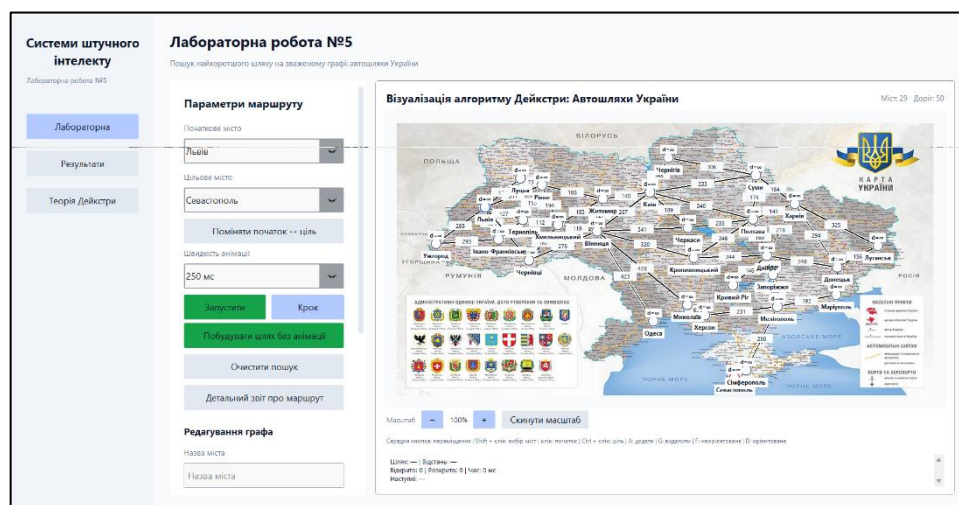


Рис.3 – Загальний вигляд інтерфейсу.

Вкладка «Лабораторна» є основним робочим середовищем програми та містить засоби налаштування параметрів експерименту, керування графом, запуску алгоритму пошуку та його візуалізації.

У секції «Параметри маршруту» користувач може вибрати початкову та цільову вершини, змінити їх місцями, запустити алгоритм покроково чи без анімації, отримати детальний звіт пошуку та редагувати граф.

Задаються початкова (Start) та цільова (Target) вершини пошуку у відповідних полях. Їх можна задати натисненням на графі правою кнопкою миші на клітинку (початок), а натисненням правою кнопкою миші та ctrl – цільову клітинку. Для проведення експериментів зі зміною напрямку пошуку передбачено функцію «Поміняти S/T», яка автоматично змінює початкову та цільову вершини місцями.

Для керування швидкістю демонстрації алгоритму передбачено параметр часу виконання одного кроку. Він використовується під час покрокової анімації

пошуку та дозволяє спостерігати послідовність розкриття вершин і зміни стану графа.

Основні операції керування пошуком представлені такими функціями (див. рис.4):

- «Запустити» — виконує пошук алгоритмом однонапрямленим хвильовим пошуком від заданої початкової вершини до цільової;
- «Крок» — виконує один етап алгоритму, що дає змогу детально простежити послідовність роботи алгоритму;
- «Побудувати шлях без анімації» — виконує пошук та відображає кінцевий результат без покрокової візуалізації;
- «Очистити пошук» — видаляє результати поточного пошуку та повертає лабіринт до початкового стану.

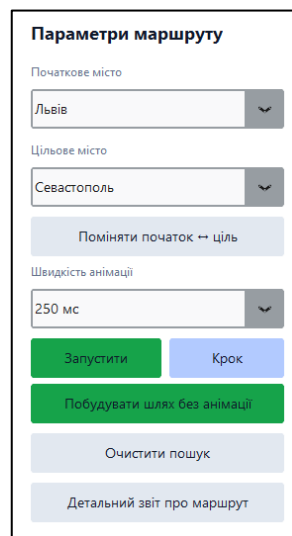


Рис.4 –Вигляд (основної частини) параметрів експерименту.

Для дослідження різних конфігурацій графа реалізовано секцію для редагування графа. За її допомогою можна змінювати структуру графа створення та видалення вершин та ребер. Для цього, окрім реалізації в інтерфейсі відповідних полів, є скорочення клавіш:

- A – додати вершину;
- G –видалити вершину;
- F – створити ребро;
- D – створити дугу.

Для швидкого повернення до початкової конфігурації передбачено функцію «Відновити граф». Вона відновлює вихідну структуру графа після виконання експериментів з зміною лабіринту.

Центральну частину робочого вікна займає область візуалізації графа. Граф представлений у вигляді структури із картою України на задньому фоні.

Під час виконання алгоритму візуально відображаються проміжні стани алгоритму (див. рис.5). Це дозволяє простежити порядок розкриття клітинок, формування черги та поступове наближення до цільової клітинки. Зміна візуального стану вершини використовується для розмежування невідвіданих, відвіданих, поточної та розкритої вершин. Після успішного завершення пошуку знайдений маршрут додатково виділяється на графі.

Такий підхід забезпечує наочне представлення алгоритму та дозволяє безпосередньо співвіднести теоретичний принцип пошуку з фактичною послідовністю обробки вершин.

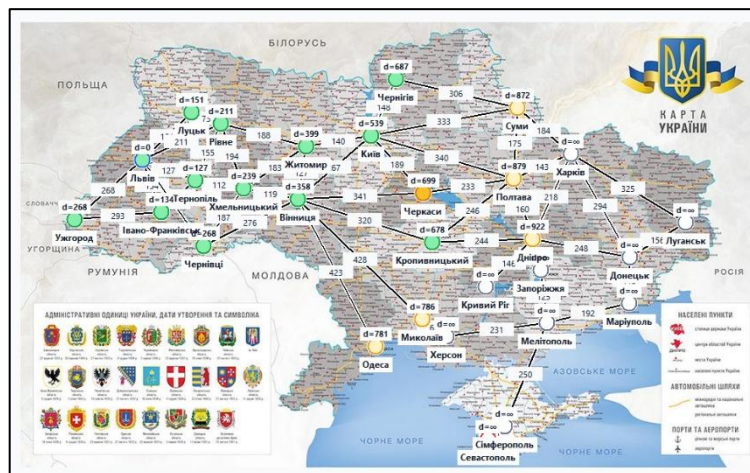


Рис.5 – Візуалізація графа

Вкладка «Результати» призначена для фіксації результатів проведених експериментів. Результати можуть зберігатися у вигляді окремих записів, що дозволяє порівнювати різні конфігурації графа.

Додавання запису до таблиці результатів здійснюється власноруч (див. рис.6).

Додати новий результат

Очистити таблицю

№	Початок	Ціль	Маршрут	Відстань (км)	Розмір	Ітерацій	Час

Рис.6 – Вкладка «Результати»

Вкладка «Теорія Дейкстри» (див. рис.7) містить стислий довідковий матеріал щодо алгоритму пошуку. У ній наведено основний принцип роботи, особливості використання, порядок розкриття вершин, умови знаходження найкоротшого шляху та основні характеристики алгоритму.

Наявність теоретичного розділу безпосередньо в програмному середовищі забезпечує можливість оперативного звернення до необхідних теоретичних відомостей під час проведення експериментів.

Теорія Дейкстри

Алгоритм Дейкстри знаходить найкоротші шляхи у зваженому графі з невід'ємними вагами. На відміну від пошуку в ширину (BFS), який вважає всі ребра однаковими та використовує чергу FIFO, Дейкстра зберігає найкращу відстань до кожної вершини й обирає наступною вершину з найменшою відстанню за допомогою мінімальної черги пріоритетів.

Основні кроки:

1. Призначити початковій вершині відстань 0, іншим — нескінченність.
2. Вилучити з черги вершину з найменшою відстанню.
3. Для кожного сусіда перевірити, чи скорочує шлях через поточну вершину відому відстань.
4. Додати покращені відстані до черги та повторювати, доки ціль не буде остаточно оброблена.

Часова складність із бінарною купою: $O((V + E) \log V)$.

Просторова складність: $O(V + E)$.

Обмеження: алгоритм коректний лише для невід'ємних ваг. Ребра з від'ємною вагою можуть порушити остаточно оброблені відстані; для таких графів потрібен інший алгоритм, наприклад Беллмана—Форда. У незваженому графі пошук у ширину зазвичай простіший і має складність $O(V + E)$.

Рис. 7 – Вкладка «Теорія»

Таким чином, розроблений графічний інтерфейс об'єднує засоби формування та редагування графа, налаштування параметрів пошуку, покрокового виконання, візуального спостереження за його роботою та фіксації отриманих результатів. Це забезпечує необхідну функціональність для проведення досліджень впливу конфігурації графа на результати роботи алгоритму пошуку.

Результати дослідження роботи програми

Дослідження 1: Базовий пошук найкоротшого шляху на схемі автомобільних шляхів України

У першому експерименті було виконано пошук найкоротшого маршруту на неорієнтованому графі з базовими вагами.

- Початкова вершина (Start): Львів
- Цільова вершина (Target): Маріуполь



Рис. 8 — Візуалізація пошуку найкоротшого маршруту від Львова до Маріуполя за алгоритмом Дейкстри

Дослідження (див. рис.8) показало знайдений маршрут: Львів → Тернопіль → Хмельницький → Вінниця → Миколаїв → Херсон → Мелітополь → Маріуполь. Сумарна довжина шляху рівна 1278 км. Кількість розкритих (зафіксованих) вершин: 26. Кількість відвіданих вершин (були в черзі): 28. Час виконання: 0.187 мс.

Алгоритм послідовно розкривав вершини відповідно до зростання сумарної відстані від Львова. Хоча існували альтернативні сполучення (наприклад, через Рівне або Івано-Франківськ), жадібний вибір пріоритетної черги спрямував пошук через Тернопіль та Хмельницький як напрямок із найменшою сумарною відстанню. Пошук завершився одразу в момент вилучення вершини «Маріуполь» із черги з пріоритетом, що дозволило не обробляти залишкові тупикові вузли півночі та Криму.

Дослідження 2: Дзеркальна зміна напрямку пошуку ($Start \leftrightarrow Target$)

Для перевірки коректності роботи алгоритму на неорієнтованому графі було проведено дзеркальну заміну початкової та кінцевої точок маршруту.

- Початкова вершина (Start): Маріуполь
- Цільова вершина (Target): Львів

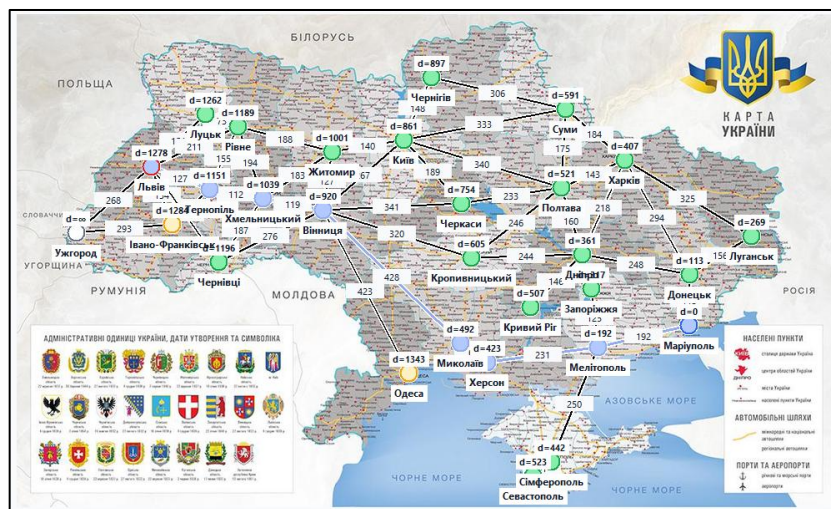


Рис. 9 — Візуалізація пошуку при заміні початкової та цільової вершин місцями

Дослідження (див. рис.9) показало знайдений маршрут: Маріуполь → Мелітополь → Херсон → Миколаїв → Вінниця → Хмельницький → Тернопіль → Львів. Сумарна довжина шляху: 1278 км. Кількість розкритих (зафіксованих) вершин: 26. Кількість відвіданих вершин (були в черзі): 28. Час виконання: 0.191 мс.

Сумарна довжина маршруту та послідовність пройдених міст виявилися строго симетричними до попереднього досліду, що підтверджує властивість симетричності матриці відстаней неорієнтованого графа. А також показники кількості розкритих вершин однакові.

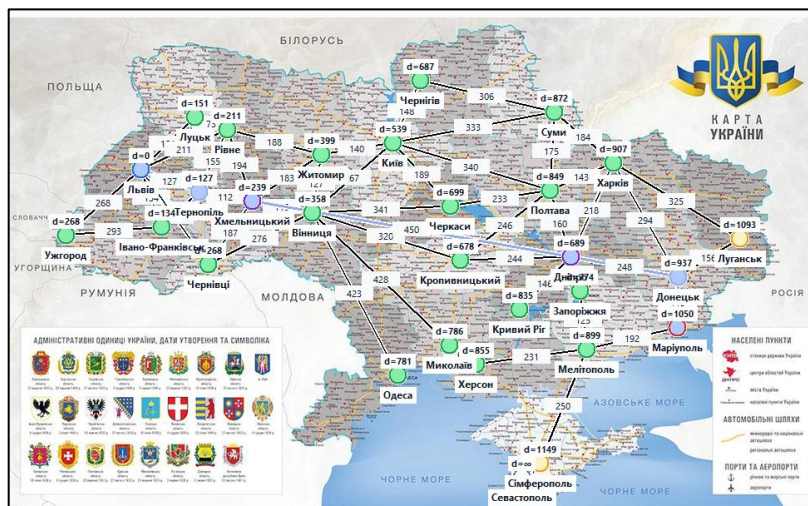


Рис.11 — Маршрут після додавання швидкісного сполучення Хмельницький – Дніпро

Дослідження (див. рис.11) показало знайдений маршрут: Львів → Тернопіль → Хмельницький → Дніпро → Донецьк → Маріуполь. Сумарна довжина шляху: 1050 км. Скорочення дистанції: 228 км. Кількість розкритих вершин: 26.

Завдяки механізму релаксації нова дуга з меншою вагою успішно оновила мітку Дніпро, витіснивши транзитні вузли Вінницю та Кропивницький із оптимального шляху.

Дослідження 4: Пошук найкоротшого шляху на орієнтованому графі (односторонній рух)

У реальних умовах деякі траси можуть мати односторонній рух або тимчасові обмеження. У цьому досліді частину ребер центрального регіону було замінено на односпрямовані дуги:

- Вінниця → Миколаїв (рух назад заборонено);
- Дніпро → Кропивницький (заборонено рух Кропивницький → Дніпро).

Було проведено пошук від Львова до Маріуполя.

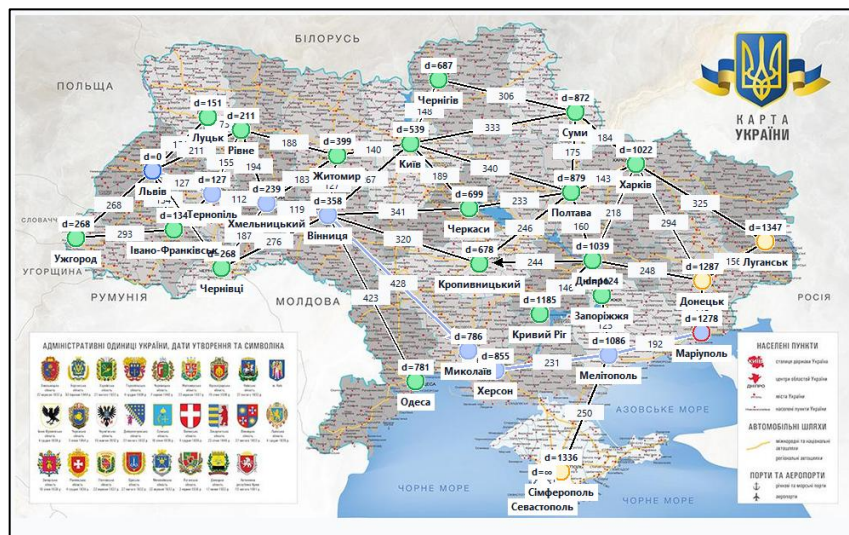


Рис. 12 — Робота алгоритму Дейкстри за наявності односторонніх обмежень

Дослідження (див. рис.12) не показало зміни відносно дослідження 1. Змінимо місцями початкову та цільову вершини.

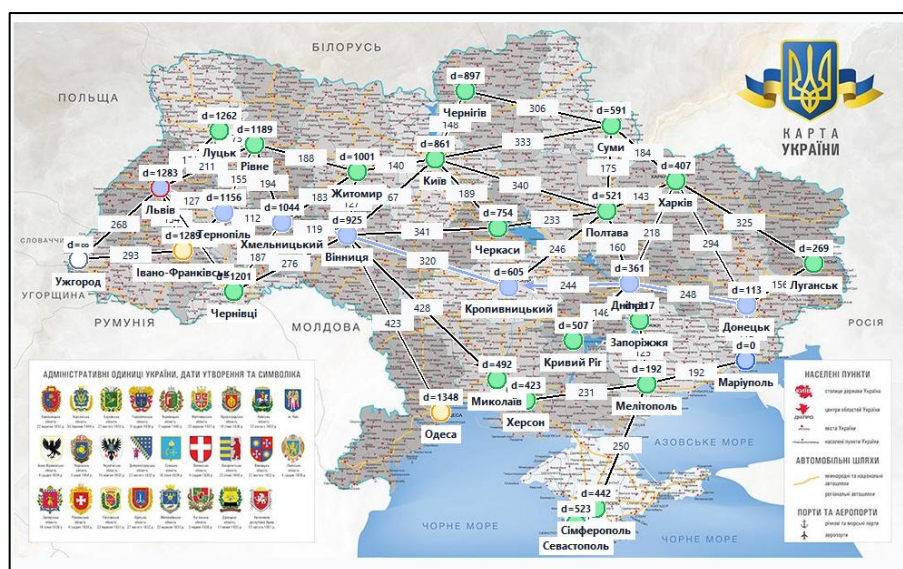


Рис. 13 — Робота алгоритму Дейкстри за наявності односторонніх обмежень при змінні початкової вершини та цільової місцями

Дослідження (див. рис.13) показало зміну маршруту, що не збігається з дослідженням 2. Знайдений маршрут: Львів → Тернопіль → Хмельницький → Житомир → Київ → Полтава → Дніпро → Донецьк → Маріуполь. Сумарна довжина шляху: 1283 км. Кількість розкритих вершин: 26. Час: 0.188 мс.

При цьому необхідно враховувати, що довжини шляхів, отримані для різних операторів, не завжди доцільно порівнювати як звичайну геометричну відстань, оскільки сам набір дозволених переходів є різним. У кожному випадку алгоритм знаходить найкоротший шлях відповідно до вибраної моделі переходів.

Висновок

У ході виконання лабораторної роботи було створено програмне забезпечення для пошуку найкоротшого шляху на зважених графах за допомогою алгоритму Дейкстри та досліджено особливості його функціонування на прикладі схеми автомобільних сполучень між найбільшими містами України. Реалізований графічний інтерфейс користувача забезпечив візуалізацію дорожньої мережі з відображенням міст і кілометражу ребер, покрокову та миттєву демонстрацію розкриття вершин, динамічне редагування топології графа (додавання й видалення міст, ребер та дуг із коригуванням їхніх ваг), а також точну фіксацію обчислювальних результатів.

Під час серії практичних експериментів було отримано такі ключові результати:

- 1) Базовий пошук на симетричній мережі: для початкової вершини «Львів» та цільової «Маріуполь» алгоритм Дейкстри визначив оптимальний маршрут через південний коридор: Львів → Тернопіль → Хмельницький → Вінниця → Миколаїв → Херсон → Мелітополь → Маріуполь із сумарною відстанню 1278 км, розкривши 26 вершин за 0.187 мс. Це підтвердило жадібний принцип алгоритму, який за рахунок черги з пріоритетом (Min-Heap) обирає переходи з найменшою поточною накопиченою вагою, відсікаючи завідомо довші альтернативи.
- 2) Дзеркальна зміна напрямку (Start $\leftrightarrow$ Target): при зміні початкової та кінцевої точок місцями (Маріуполь → Львів) алгоритм зафіксував строго симетричний маршрут ідентичної довжини (1278 км), обробивши однакову кількість вершин (26 розкритих та 28 відвіданих). Це експериментально довело інваріантність результату та коректність роботи алгоритму на неорієнтованих графах із симетричною матрицею ваг.
- 3) Адаптивність до зміни порядку та розміру графа:
 - a. Вилучення транзитного вузла: видалення ключового міста «Вінниця», через яке проходив базовий шлях, змусило алгоритм за допомогою релаксації суміжних ребер перебудувати рух через північний коридор (Львів → Рівне → Житомир → Київ → Полтава → Дніпро → Донецьк → Маріуполь). Це збільшило дистанцію на 122 км (до 1400 км), продемонструвавши здатність алгоритму автоматично знаходити найкращий альтернативний об'їзд при порушенні зв'язності окремих ділянок.
 - b. Додавання нової магістралі: введення швидкісного сполучення Хмельницький – Дніпро (450 км) дозволило виключити транзит через Вінницю та південні/центральні вузли, скоротивши загальну відстань між Львовом та Маріуполем на 228 км (до 1050 км).
- 4) Вплив орієнтованості зв'язків (одностороннього руху): заміна частини ребер на напрямлені дуги (Вінниця → Миколаїв та Дніпро →

Кропивницький) у прямому напрямку Львів $\rightarrow$ Маріуполь не змінила маршрут (1278 км), оскільки рух збігався з дозволеним вектором дуги. Проте при зворотній спробі проїзду з Маріуполя до Львова одностороннє обмеження заблокувало південну трасу, внаслідок чого алгоритм знайшов інший допустимий шлях через центральні області (через Дніпро, Полтаву, Київ, Житомир, Хмельницький та Тернопіль) довжиною 1283 км за 0.188 мс. Це засвідчило, що на орієнтованих графах найкоротші відстані між парою вершин u та v перестають бути симетричними ($d[u \rightarrow v] \neq d[v \rightarrow u]$).

Теоретичний і практичний аналіз показав, що алгоритм Дейкстри є метричною модифікацією пошуку в ширину (BFS). Заміна простої черги FIFO на чергу з пріоритетом і впровадження операції релаксації ребер дозволяють алгоритму поширювати фронт пошуку не за кількістю ребер, а концентрично за сумарною вагою.

Поставлену мету лабораторної роботи повністю досягнуто: розроблено працездатний програмний комплекс, побудовано топологічну модель автомобільних шляхів України та всебічно досліджено особливості знаходження найкоротших шляхів за алгоритмом Дейкстри за різних конфігурацій транспортної мережі.

Код програми

Код до даної лабораторної роботи розміщений у репозиторії https://github.com/MrCatman137/AI_systemem у директорії lab_5 та має повну інструкцію запуску даної програми.